

CISC 365

Project 2

Dynamic Programming

Fall 2022

Deadline: Nov 30, 2022 11:59 PM

Please include the names and student numbers of your team members in your submission.

Submit two Python files T4-1.py and T4-2.py with your answers to the two experiments. Please implement your algorithm and provide the corresponding pseudo-code and time complexity analysis.

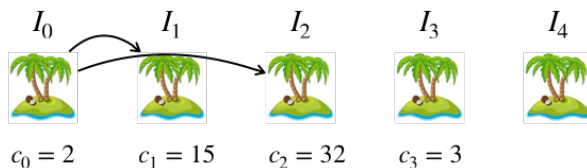
Your implementation should be able to run on the provided test code.

Experiment 1 (25 Marks)

Island Hopping: In the example below, we want to get to the destination island I_4 from I_0 by hopping from left to right. Each time we can either hop one island or two islands. Leaving an island i is associated with a cost c_i , and we need to pay the cost in order to hop from it. At I_0 , once we pay the cost c_0 , we can either get to I_1 or to I_2 in one hop. For instance, we can hop $I_0 \rightarrow I_1 \rightarrow I_3 \rightarrow I_4$ and the total cost is $c_0 + c_1 + c_3$.

Our goal is to find a hopping path to get to I_4 while paying the lowest total cost.

Provide a $O(n)$ algorithm for the generalized version of this problem, *island_hopping*(c), that computes the minimal cost of getting to the destination island I_n . Parameter c is the cost array; $n = |c|$ is the index of the destination island. Your function should **return both the minimum cost and an optimal hopping path**. For the example below, *island_hopping*([2, 15, 32, 3]) returns (20, '0-1-3-4')



More examples,

island_hopping([15, 3, 11, 36]) returns (26, '0-2-4')

island_hopping([10, 10, 40, 33, 15, 1]) returns (54, '0-1-3-5-6')

island_hopping([15, 3, 11, 36, 2, 18]) returns (28, '0-2-4-6')

Experiment 2 (25 Marks)

Pacman: Help Pacman calculate the total number of possible paths getting to the cherries. Pacman starts at $(0,0)$ and can only move along the lines in two directions, right or down. There are ghosts blocking certain locations. The presence of ghosts is given as a $n \times n$ matrix $ghost$, and $ghost_{ij} = 1$ if there is a ghost blocking the corresponding location (i,j) , otherwise $ghost_{ij} = 0$.

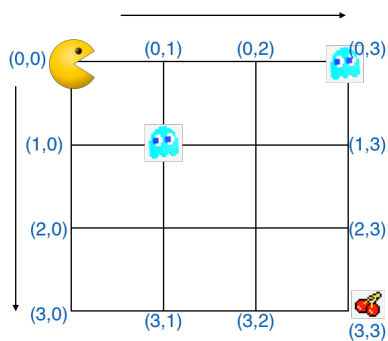
Provide a $O(n^2)$ algorithm for the generalized version of this problem, $pacman(g)$, that inputs a 2-d list and **returns the total number of possible paths getting to the cherries**. Parameter $ghost$ is the ghost location matrix and $(n-1, n-1)$ is the location of the cherries.

For the example pictured below,

$pacman([[0,0,0,1], [0,1,0,0], [0,0,0,0], [0,0,0,0]])$ returns 7

Another example

$pacman([[0,1], [0,0]])$ returns 1



Marking Scheme (for both questions):

Program is correct and must use dynamic programming (10 points)

- Program is correct but does not use DP 4 points
- Program is correct, uses DP, but fails on few testing cases 4 points
- Program is correct, uses DP, but fails on many testing cases 2 points
- Program does not run, or fails on most testing cases 1 point

Program has the required efficiency (6 points)

- Program only fails on few test cases but meets the efficiency 5 points
- Program is correct but slightly inefficient 3 points
- Program is correct but significantly inefficient 1 point

Pseudocode is in plain English and is a clear description of your algorithm (3 points)

- Using only slightly modified Python program for pseudocode 1 point

Parameters, return, and function name are as specified (2 points)

Program style and logic are clear and concise (2 points)

Explanation on the complexity of the algorithm is clear (1 point)

Comments are appropriate and program is easy to understand (1 point)